پروژه پایانی دوره جاوااسکریپت 15

📚سیستم مدیریت کتابخانه📚

تاریخ تدوین: شهریور ۱۴۰۵

در این پروژه، شما یک سیستم مدیریت کتابخانه کامل پیاده‌سازی می‌کنید. بک‌اند این پروژه از قبل پیاده‌سازی شده و روی سرور قرار گرفته است. از تمام چیزهایی که در طول دوره یادگرفتید می‌تونید استفاده کنید. نکاتی که توی تمرین‌ها داشتیم رو حتما رعایت کنید.

آنچه باید انجام دهید

شما باید یک رابط کاربری کامل برای سیستم مدیریت کتابخانه بسازید که شامل موارد زیر باشد:

1. صفحه ورود - کاربران بتوانند وارد سیستم شوند
2. داشبورد دانشجو - نمایش آمار و اطلاعات شخصی
3. صفحه کتاب‌ها - نمایش لیست کتاب‌ها و امکان امانت گرفتن
4. صفحه امانت‌های من - مدیریت کتاب‌های امانت گرفته شده
5. مدیریت احراز هویت - ورود، خروج و حفظ وضعیت کاربر

شروع کار

1. دریافت فایل‌های پروژه

ابتدا فایل‌های قالب پروژه را از مخزن زیر کلون کنید:

```
git clone git@github.com:karyar-js11/library-management-system-template.git
```

```
cd library-management-system-template
```

در این پروژه، فایل‌های html , css از قبل پیاده‌سازی شده. می‌توانید به صلاحدید خودتون اونارو تغییر بدید.
لینک: https://github.com/karyar-js11/library-management-system-template/tree/main

2. آدرس API

بک‌اند پروژه از طریق آدرس زیر در دسترس است.

آدرس پایه: https://haditabatabaei.dev/api

آدرس مستندات: https://haditabatabaei.dev/api/docs

3. ساختار فایل‌ها

پس از کلون کردن، ساختار فایل‌ها به این صورت خواهد بود:

```
project-name/
├── css/
│   └── style.css        # استایل‌های آماده
├── js/                  # پوشه کدهای JavaScript شما
├── images/              # پوشه تصاویر
├── index.html           # صفحه اصلی
├── login.html           # صفحه ورود
├── dashboard.html       # داشبورد دانشجو
├── books.html           # صفحه کتاب‌ها
├── my-loans.html        # صفحه امانت‌های من
```

توضیحات پروژه # README.md └──

کاربران موجود در سیستم و اطلاعات ورود

برای هر کدام از شما یک یوزرنیم و پسورد (که هردو ایمیل شماست) ساخته شده است. دقت کنید که پایگاه داده‌ی کتاب‌ها برای تمام شما مشترک است. به این معنا که اگر کتابی اضافه/کم/قرض گرفته شود و یا پس داده شود، نتیجه‌ی آن را همگی خواهید دید. این رفتار، مشابه پروژه‌های واقعی است.

- نام کاربری: آدرس ایمیل شما که به کاریار داده‌اید.
- رمز عبور: آدرس ایمیل شما که به کاریار داده‌اید.

داده‌های موجود

- تعدادی کتاب
- دانشجویان با اطلاعات کامل

API های موجود

احراز هویت

- POST /api/auth/login - ورود کاربر
- GET /api/auth/me - دریافت پروفایل و آمار کاربر

کتاب‌ها

- GET /api/books - دریافت لیست تمام کتاب‌ها
- GET /api/books/:id - دریافت جزئیات یک کتاب

امانت‌ها

- GET /api/loans/my-loans - دریافت امانت‌های کاربر

- POST /api/loans - امانت گرفتن کتاب
- POST /api/loans/:id/return - بازگرداندن کتاب

نحوه استفاده از API

برای استفاده از API ها، باید درخواست‌های HTTP ارسال کنید. تمام درخواست‌ها (به جز ورود) نیاز به توکن JWT دارند که باید در header ارسال شود. این هدر باید به اسم authorization باشد و مقدار آن رشته‌ی "Bearer YOUR_TOKEN" باشد. توکن JWT یک رشته است که به عنوان شناسه‌ی شما، بین کلاینت (مرورگر) و سرور رفت و آمد دارد. از طریق این شناسه، سرور هویت شما را تشخیص می‌دهد که لازم نباشد هربار رمزعبور خود را ارسال کنید. برای دیدن جزئیات هر درخواستی که بالاتر معرفی شد، می‌توانید از لینک مستندات api در قسمت آدرس API استفاده کنید.

🛠️ وظایف شما

1. صفحه ورود (login.html)

- فرم ورود با فیلدهای ایمیل و رمز عبور
- ارسال درخواست به API برای ورود
- ذخیره توکن JWT در کوکی (بدون HttpOnly)
- هدایت به داشبورد پس از ورود موفق
- نمایش پیام خطا در صورت ورود ناموفق

فرآیند ورود:

1. کاربر ایمیل و رمز عبور را وارد می‌کند
2. درخواست POST به /api/auth/login/ ارسال می‌شود
3. در صورت موفقیت، توکن JWT دریافت می‌شود
4. توکن در کوکی ذخیره می‌شود (بدون HttpOnly)
5. کاربر به صفحه داشبورد هدایت می‌شود

2. داشبورد دانشجو (dashboard.html)

- نمایش نام کاربر
- نمایش آمار (تعداد امانت‌های فعال، کتاب‌های موجود)
- دکمه‌های دسترسی سریع
- بارگذاری اطلاعات از API

3. صفحه کتاب‌ها (books.html)

- نمایش لیست کتاب‌ها در قالب کارت
- نمایش اطلاعات: عنوان، نویسنده، ISBN، دسته‌بندی، تعداد موجود
- دکمه امانت گرفتن برای کتاب‌های موجود
- دکمه غیرفعال برای کتاب‌های ناموجود
- کش کردن نتایج API برای 5 دقیقه به طوری که با رفرش صفحه از بین نرود

4. صفحه امانت‌های من (my-loans.html)

- نمایش لیست امانت‌های کاربر
- نمایش اطلاعات کتاب و تاریخ امانت
- دکمه بازگرداندن کتاب
- آمار امانت‌ها

5. مدیریت احراز هویت

- بررسی وجود توکن در کوکی
- هدایت به صفحه ورود در صورت عدم وجود توکن
- اضافه کردن توکن به درخواست‌های API
- خروج از سیستم و پاک کردن توکن

6. حفاظت از صفحات

- کاربران غیروارد: فقط می‌توانند صفحات index.html و login.html را ببینند
- کاربران وارد شده: نمی‌توانند login.html را ببینند و باید به داشبورد هدایت شوند

- بررسی وضعیت ورود: در هر صفحه بررسی شود که کاربر وارد شده است یا نه

📌 نکات مهم

احراز هویت

- تمام درخواست‌های API (به جز ورود) نیاز به توکن JWT دارند
- توکن را در header با نام Authorization و مقدار Bearer <token> ارسال کنید
- توکن را در کوکی ذخیره کنید به طوری که از طریق جاوااسکریپت به آن دسترسی داشته باشید.

مدیریت خطا

- همیشه پاسخ API را بررسی کنید
- در صورت خطا، پیام مناسب در جای مناسب نمایش دهید
- وضعیت loading را برای کاربر نشان دهید

نکات فنی

ذخیره توکن در کوکی:

- از document.cookie برای ذخیره و بازیابی توکن استفاده کنید
- در هر درخواست HTTP، توکن را از کوکی بخوانید و در header ارسال کنید

کش کردن لیست کتاب‌ها:

- نتایج API کتاب‌ها را برای 5 دقیقه کش شوند. این کش نباید با رفرش صفحه از بین برود.
- قبل از درخواست جدید، ابتدا بررسی کنید که آیا داده‌های کش شده معتبر هستند یا نه
- در صورت منقضی شدن کش، درخواست جدید ارسال کنید

حفاظت از صفحات:

- در ابتدای هر صفحه (به جز index و login)، بررسی کنید که کاربر وارد شده است

- اگر کاربر وارد نشده، به صفحه login هدایت کنید
- اگر کاربر وارد شده و در صفحه login است، به داشبورد هدایت کنید

معیارهای ارزیابی

عملکرد (45%)

- صحت پیاده‌سازی API و اتصال به بک‌اند
- مدیریت احراز هویت و کوکی‌ها
- مدیریت خطاها و استثناها
- امنیت و اعتبارسنجی ورودی‌ها
- عملکرد و بهینه‌سازی کد
- پیاده‌سازی کش و localStorage
- حفاظت از صفحات و routing
- مدیریت state و data flow
- صحت عملکرد تمام ویژگی‌ها

معماری کد (35%)

- ساختار و سازماندهی کد
- استفاده از بهترین روش‌های JavaScript
- تقسیم‌بندی منطقی فایل‌ها و توابع
- خوانایی و قابل نگهداری بودن کد
- استفاده صحیح از async/await و Promise
- مدیریت رویدادها و event handling
- پیاده‌سازی الگوهای مناسب JavaScript

رابط کاربری (15%)

- طراحی و ظاهر صفحات

- تجربه کاربری (UX)
- سازگاری با مرورگرهای مختلف

ویژگی‌های جدید (5%)

- نوآوری و خلاقیت
- ویژگی‌های اضافی و بهبودها
- بهینه‌سازی‌های شخصی

تحویل پروژه

فایل‌های مورد نیاز

- تمام فایل‌های HTML
- فایل‌های JavaScript در پوشه js/
- فایل‌های CSS
- فایل README.md با توضیح پروژه

نحوه تحویل و کار با گیت

۱. ایجاد مخزن Git

- یک مخزن خالی در GitHub/GitLab ایجاد کنید
- نام مخزن: library-management-frontend یا مشابه
- مخزن باید عمومی (Public) باشد. این آدرس را برای ما ارسال کنید.

۲. کار با Git

- ابتدا قالب‌ها را از مخزن اصلی کلون کنید
- سپس مخزن را به مخزن شخصی خودتان متصل کنید
- فایل‌ها را اضافه کرده و کامیت اولیه ایجاد کنید

- تغییرات را به مخزن شخصی خود push کنید.
- هر تغییر مهم را کامیت کنید
- پیام‌های کامیت واضح و توصیفی بنویسید
- به طور منظم تغییرات را push کنید

۳. تحویل نهایی

- لینک مخزن عمومی خود را ارسال کنید
- در README.md توضیح دهید که چه کاری انجام داده‌اید
- مطمئن شوید که آخرین تغییرات push شده‌اند

موفق باشید! 🚀

برای علاقه‌مندان، سورس کد پروژه‌ی بکند در این آدرس قابل دسترسی است.